

console-next — P0 (Foundation + one vertical)

A thin, working vertical of the console UX redesign (see `docs/CONSOLE-UX-REDESIGN.md`). It **coexists** with the existing consoles (`static/*-console.html` and their routes are untouched) — this is **additive only**. New route: `/console-next/`.

Stack (per §16): **Lit (Web Components) + design tokens (CSS custom properties)**, **no build step**, **vendored Lit** (no runtime CDN), Shadow DOM per component, base-path-aware.

What's implemented (works today)

1. Persistent shell (§4) — Lit components

- **Status bar** — workspace name, cloud-lens switcher (stub `<select>`), endpoint **click-to-copy**, conformance pill (placeholder), TTL chip + reset (stubs), `%I` Inspector toggle, `%K` palette.
- **Left service rail** — searchable; renders **only** the services in the capability manifest's catalog (§8, catalog-driven).
- **Center canvas** — routes to a widget by the service's `widget` id + its capability **mode** (data, not substrate branching).
- **Right Inspector drawer** — four tabs; Calls is live (below).
- **%K command palette** — jump-to-service + toggle-Inspector + a "reveal CLI" placeholder (F8).

2. Capability manifest (§15.1) — `GET /api/console/capabilities`

New appliance endpoint returning the runtime descriptor: `substrate`, `cloud_lenses`, `widgets` (per-widget mode), `features`, `connect.mode`, plus the service catalog + workspace info. **The UI reads this and gates on it** — there is **no `if (substrate===...)` / `if (cloud===...)` branching in any component** (§15.2). Substrate differences are one field here (e.g. `connect.mode`, `features.ssh`).

3. Connect-contract component (§13.1) — `components/connect-contract.js`

One reusable component with the three invariant blocks **Connect / Live / Actions**, parameterized by resource via properties + named slots. The `connect.mode` variant (`endpoint` / `ssh` / `relay`) only changes the Connect block's copy — it is **not** three components. The **● backed by <X> badge is mandatory** and always rendered.

4. object-browser widget wired to S3 (§13.3) — `widgets/object-browser.js`

Real, against the appliance's existing `/api/s3` REST API (backed by **real MinIO** on the local stack):

- **list buckets** (`GET /api/s3/buckets`)
- **create bucket** (`POST /api/s3/buckets/{name}`)
- **list objects** (`GET /api/s3/buckets/{b}/objects`)

- **view an object** — text / hex / image preview from **real bytes** (`.../objects/{key}/meta` + `.../download`)
- **upload a file** — multipart to `POST .../objects`
- Rendered inside the Connect contract with the **backed by MinIO** badge and a boto3 / CLI / TF snippet + `reveal CLI` line.

5. Glass-box Inspector — Calls tab (§12) — `core/glassbox.py` + Inspector

- **ASGI capture middleware** (`GlassBoxCaptureMiddleware`) added in `server.py` as the **outermost** layer. It taps every request/response, resolves `cloud/service/action` (host + path + `X-Amz-Target` + `Query Action`), stamps latency, and pushes the **exact §12.2 event** onto an in-memory **ring buffer** (default 500; `VYOMI_GLASSBOX_RING`).
- **Redaction at tap time (§12.7):** `Authorization` , `x-amz-security-token` , license headers, cookies are `<redacted>` ; only the SigV4 **access-key id** is kept as `principal` — never the secret / KMS plaintext / secret values.
- **Explainer (§12.5):** deterministic, offline map of error codes → plain-language why + hint (`NoSuchKey` , `NoSuchBucket` , `BucketNotEmpty` , `AccessDenied` , ...) with a generic-status fallback.
- **backend.ref seam (§12.4):** each S3 call carries `{kind: minio, served_by: s3_core, ref:{type, bucket, key}}` . The object key is recovered from the URL, or from the JSON upload body for the collection-POST.
- `GET /api/console/calls/stream` — SSE endpoint (backlog replay + live) that the Lit Inspector consumes via `EventSource` .
- `GET /api/console/calls` — one-shot buffer snapshot; `POST .../clear` — clear the buffer.
- The Inspector renders the live timeline (method-colored, status-colored, latency), filter, pause/resume, click-to-expand (request/response summaries + the explainer), and **view in backend** ▶ which deep-links an S3 call to that bucket/object in the object-browser.

6. New route — `/console-next/`

- `GET /console-next[/...]` serves the SPA `index.html` (client-side router owns sub-paths).
- Static assets mounted at `/console-next-assets/` . **Base-path-aware:** the asset base is derived from `location.pathname` at runtime, so the same bundle works at web root and under a subpath (Nano `/nano/` scope, §15.1 #6).

What's stubbed / deferred (by design, per §14.6 P0 scope)

- **cloud-lens switcher, conformance pill, TTL/reset** — visual stubs (live conformance panel = F5/P1).
- **Inspector State / Snapshots / Conformance tabs** — placeholders (State deep-read = P1; snapshot/fork/replay = P2). The **Calls** tab is fully live.
- **Non-S3 services** (DynamoDB, RDS, IAM in the catalog) render the **generic-control-plane** fallback — the honest conformance gate (§14.5); their rich widgets land in P1.
- `reveal CLI` , `snapshot` , `fork` , `replay` buttons — present but disabled/stubbed.
- **Familiar** ▼ native lens — visible, inert (theming layer F9, deferred).
- **multipart upload deep-link key** — the collection-POST multipart upload does not expose the key in `backend.ref` (only the JSON path does); the object's subsequent `HeadObject/GetObject` calls carry the

full key ref, so the deep-link still works.

How to run / serve it

The console is served by the appliance FastAPI app (`server.py`). Boot the stack the usual way (needs Docker/MinIO for the S3 data plane to be real):

```
# from the appliance repo root (the stack brings up MinIO etc.)
docker compose -f docker-compose.appliance.yml up # or your usual `vyomi up`
```

Then open:

```
http://localhost:<appliance-port>/console-next/
```

(If serving under a subpath / Nano scope, the asset + API bases adjust automatically.)

Optional env knobs:

- `VYOMI_GLASSBOX_RING` — ring buffer size (default 500)
- `VYOMI_GLASSBOX_BODY_CAP` — bytes summarised per body (default 2048)
- `VYOMI_SUBSTRATE` — `local` | `codespaces` | `nano` (default `local`)
- `VYOMI_CONNECT_MODE` — `endpoint` | `ssh` | `relay` (default `endpoint`)
- `VYOMI_WORKSPACE_NAME` , `VYOMI_S3_ENDPOINT` — status-bar / snippet values

The 3-chord demo (one service)

1. Open `/console-next/` → S3 is selected; the Connect card shows the endpoint + snippet +  backed by MinIO.
2. Point real boto3 / `aws --endpoint-url ...` at the endpoint (or use the in-console create-bucket + upload). Watch calls stream into the **Inspector** → **Calls** live.
3. `HeadObject` a missing key → the Inspector shows the **404 + plain-language why**; click **view in backend** ▶ on a `PutObject` / `HeadObject` → jump to the real object bytes in the object-browser.

What still needs the live stack to validate

Self-verified here (no Docker): `python3 -c "import ast; ast.parse(...)"` on `server.py` + new modules; standalone glassbox unit behavior (ULID monotonicity, service/action resolution, redaction, explainer, ring buffer, `backend.ref` key recovery); `TestClient` integration proving `/api/console/capabilities` returns valid JSON, `/console-next/` serves the SPA, the vendored Lit + assets serve 200, and the **full S3** → **glass-box capture loop** (CreateBucket/PutObject/ListObjects/HeadObject incl. a 404 with the explainer) is captured with correct `backend.ref`; `node --check` on every JS module.

Needs the live stack (Docker + MinIO + a real browser):

- **Real-browser render** of the Lit SPA (no headless Chromium in this env) — the shell, rail, Connect card, Inspector SSE stream, ⌘K palette.
- **Real MinIO durability** of uploaded bytes + concurrent reach by an external S3 SDK/CLI at the same endpoint (the data-plane proof).
- **SSE over a real HTTP server** (the `TestClient` blocks on the streaming generator; the endpoint + content-type are correct, the live stream is exercised in-browser via `EventSource`).

Files added / changed

Added (backend):

- `core/glassbox.py` — event schema, ring buffer, resolver, redaction, explainer, `backend.ref`, capture middleware, SSE generator.
- `routes/console_next.py` — capabilities endpoint, calls snapshot/stream/clear, SPA route + `/console-next-assets` mount.

Added (frontend, `static/console-next/`):

- `index.html`, `boot.js`, `api.js`, `app.js`, `tokens.css`
- `vendor/lit-core.min.js` (vendored Lit 3, ~15 KB, offline)
- `components/`: `status-bar.js`, `service-rail.js`, `center-canvas.js`, `connect-contract.js`, `inspector-drawer.js`, `command-palette.js`
- `widgets/`: `object-browser.js`, `generic-control-plane.js`

Changed:

- `server.py` — registered `console_next` route module; added `GlassBoxCaptureMiddleware` as the outermost middleware. **No existing route, console, or handler modified.**

Doc: this file.

Nano substrate (P-Nano)

Nano is **not a second console** — it is a third *substrate* for this same console-next engine (§14.8, §15). The SPA talks only to `/api/*`; on Nano that API is served by an in-tab **Service Worker** → **Pyodide cores** stack instead of FastAPI. The only thing that differs is one field in the capability manifest — no `if (substrate)` anywhere in widget code (§15.1 guarantee #2, §15.2 anti-fork).

How the engine runs on Nano

- The **same bundle** (`static/console-next/`) is served under the Nano `/nano/` SW scope (base-path-aware, §15.1 guarantee #6). The SW intercepts `BASE + /api/*`, strips the prefix, and dispatches to the Pyodide cores (`aws_core_adapter` / `aws_wire_router` + the vendored cores).
- The cores are the same proven in-WASM stores used by the Nano relay bundle: `ObjectStore` (S3), `NoSqlStore` (DynamoDB), `KvStore` (Secrets), `KmsEngine` (KMS), `MessagingStore` (SQS/SNS),

SqlStore→**PGLite** (real Postgres in WASM) / sqlite.

- External SDK/CLI clients reach Nano via the **relay** (already built): the console and the relay front-door share the same cores/state in the tab.

Capability degradations (data, not screens)

GET /api/console/capabilities?substrate=nano returns:

- `connect.mode = "relay"` and `features.ssh = false`, `features.relay = true`.
- `widgets["compute-terminal"] = "degraded"` — no VM/SSH in a browser tab; the SAME `generic-control-plane` fallback renders the `degrade_notes["compute-terminal"]` CTA ("open a Codespace to SSH").
- `widgets["serverless-invoke"] = "partial"` — invoke runs in Pyodide, but there is no container deploy; `degrade_notes` carries that copy.
- **Every data-plane widget stays full** (object / sql[PGLite] / nosql / queue / kv / kms / generic) — their WASM equivalents exist (§14.9).
- AWS is the rich lens on Nano; GCP/Azure fall back to the generic control-plane (console-CRUD), consistent with today. Lens switch is unchanged (still a manifest re-fetch).

The substrate is chosen by `?substrate=` (or env `VYOMI_CONSOLE_SUBSTRATE` / `VYOMI_SUBSTRATE`); unknown values fall back to `local`. The status-bar shows a small `• nano` / `• local` / `• codespaces` tag read straight from the manifest.

Snapshot / fork on Nano

`core/console_snapshots.py` runs **unchanged** on Nano: it walks a table of `store()` accessors that resolve to the in-WASM stores in the tab; their in-process state is the identical JSON-serialisable dict shape, so capture/restore/fork need no browser and no substrate branch. Nano is actually the *lighter* snapshot substrate (§14.9). The one known gap is substrate-independent: the SQL **data plane** (live rows in sqlite3/PGLite) is a later fidelity slice — only the control-plane metadata is captured today (documented in `_dump_sql`); on Nano the full-data capture would be a PGLite dump.

Remaining live-stack integration work (needs the live Nano stack)

Scaffolded + specified here; **not runnable in this environment** (needs the browser SW bundle + Pyodide + PGLite). These are the P-Nano deliverables that must run on the live stack:

- **Wire the console-next bundle into the Nano SW bundle** under the `/nano/` base path — serve `static/console-next/` from the Nano SW alongside the relay endpoint; confirm the fetch shim rewrites the console's root-absolute `/api/*` to `BASE + /api/*` (guarded to not double-prefix already-mounted `/nano/` paths), and retire the old per-service Nano consoles.
- **SW-side glass-box interceptor** — a Service-Worker/router tap that emits the **identical §12.2 event schema** as the FastAPI `GlassBoxCaptureMiddleware`, so the Inspector Calls tab is byte-for-byte the same across substrates (§15.1 guarantee #5). Nano is the easier capture seam (one in-tab router) — §14.9.
- **Real-browser render** of the SPA served by the Nano SW: shell / rail / Connect card (relay variant) / Inspector SSE / ⌘K, plus the degrade CTA on compute-terminal and serverless-invoke, and object/sql/nosql/queue/kv/kms CRUD through the cores.

- **Relay-variant Connect proof** — external SDK/CLI hitting the relay endpoint reads the same tab state the console mutates (one shared engine); PGlite (real Postgres) reachable in-tab + via the RDS Data API over the relay.
- **Cross-substrate parity test in CI** (§15.3) — assert the component tree + glass-box event shape are identical across local / codespaces / nano.